

Supporting Data and Code: Reproducibility Documentation

Evaluating LLM-Based Credit Rating Systems: A Pre-Registered Specification-Curve Analysis

Companion to the manuscript submission

1. Purpose and scope

This document describes the supporting data-and-code archive that accompanies the manuscript. The archive is a self-contained, **offline**, **deterministic** reproduction: from a single command it regenerates every reported number, table, and figure from frozen inputs, with **no network access, no API keys, and no paid model calls**. The permanent citable version is the Zenodo archive at 10.5281/zenodo.21953934 (code MIT-licensed); the same tree is mirrored at <https://github.com/samirrc2/llm-credit-rating-specification-curve>.

A strict wall separates two phases. An *offline analysis* reproduces every claim from frozen inputs (documented here). A *live capture* phase originally queried commercial model APIs and is retained for transparency only (Section 8). Because commercial models change over time, capture is deliberately not part of the reproducible core; the frozen model-response corpora it produced are shipped and hash-verified. The confirmatory data are synthetic firm profiles benchmarked to an objective Altman Z'' reference; the real-firm arm uses only publicly available SEC filings, anonymized and perturbed.

2. Quick start

```
pipinstall-rrequirements.txt
bashreproduce.sh
```

Expected result: the full analysis regenerates every `results/results_*.json`; a second independent run is byte-identical (the SHA-256 of each JSON result matches), and the run exits 0. Total runtime is a few minutes on a standard laptop. No network access is required at any point.

3. Archive layout

<code>reproduce.sh</code>	one-command offline reproduction of all results
<code>requirements.txt</code>	pinned Python dependencies (analysis only; Python 3.10)
<code>config/</code>	specification grid, paraphrase templates, rating scale, run config
<code>analysis/</code>	all analysis code (Section 5)
<code>capture/</code>	live data-collection harness (reference only; not run here, Section 8)
<code>data/frozen/main/</code>	pre-registered inputs: <code>battery_90.json</code> , <code>grid_definition.json</code> , <code>paraphrase_templates.json</code> , <code>rating_scale.json</code> , <code>capital_map.json</code> , manifests
<code>data/frozen/realarm/</code>	real-firm arm artifacts: anonymized battery, sealed crosswalk, rating provenance, 12-spec grid

data/frozen/market_data/	FRED option-adjusted-spread series (cached)
data/raw/main/	frozen response corpus (per-call JSON)
data/raw/realarm/	real-firm arm model-output panels (arm / comparator / fingerprint)
capture/flagship_runs/	flagship model-tier arm corpus (1,620 per-call JSON), fixed by MANIFEST_FLAGSHIP
data/panel/panel.parquet	analysis panel, built once from raw (the confirmatory analysis input)
results/	all generated outputs (Section 6)
paper/ISWA/	manuscript (Elsevier CAS), figures, highlights, cover letter, .docx
manifest/	SHA-256 manifests for every frozen stage
docs/	reports, appendix, changelog
PREREGISTRATION.md, PREREGISTRATION_AMENDMENTS.md	frozen pre-registration and amendments

4. Data (inputs and metadata)

4.1 Frozen inputs

Every input the pipeline reads is a frozen artifact. The analysis is a pure, seeded function of these files.

File	Rows	Unit of observation / key fields
data/frozen/main/battery_90.json	90	one synthetic firm profile; 45 credit-health items (SAFE/WATCH/DISTRESS, 15 each) with an Altman Z'' benchmark band + 45 directional items (buy/hold/sell). Fields: <code>item_id</code> , <code>family</code> , <code>benchmark_label</code> , <code>facts_kv</code> , <code>facts_text</code>
data/raw/main/run_20260707_185649/	12,960	one row = one model call across the pre-registered grid; the confirmatory analysis uses the two providers with complete current+prior coverage (OpenAI, Google) = $90 \times 32 \times 3 = 8,640$ cells. An excluded third-provider pilot is retained for transparency. Fields: <code>spec_axes</code> , <code>item_id</code> , <code>seed</code> , <code>provider</code> , <code>model</code> , <code>raw_response</code>
data/frozen/realarm/battery_real45.json	31	anonymized, perturbed real B2B issuers with disclosed agency ratings; benchmarked in <code>ratings_provenance.csv</code> (verbatim 10-K quote, SEC URL, document SHA-256)
data/frozen/realarm/spec_grid_12.json	12	the frozen 12-specification grid (presentation $\times$ provider $\times$ paraphrase $\times$ format) shared by the real-firm and flagship arms
capture/flagship_runs/run_20260830_141844/	1,620	flagship model-tier arm: <code>gpt-5.4</code> + <code>gemini-3.1-pro-preview</code> on the 12-spec grid $\times$ 45 credit items $\times$ 3 seeds
config/grid_definition.json	—	the balance-constrained D-optimal specification grid

File	Rows	Unit of observation / key fields
data/frozen/main/rating_scale.json	—	21-grade letter scale and the letter→notch/band/IG-HY maps
data/frozen/main/capital_map.json	—	Basel CRE20 ECRA risk-weight map (quote-verified; post-hoc)

4.2 Provenance and integrity

`manifest/` holds a SHA-256 manifest for every frozen stage: `MANIFEST_INPUTS` (pre-registered inputs), `MANIFEST_RAW` (response corpus), `MANIFEST_PANEL` (analysis panel), `MANIFEST_ANALYSIS`, `MANIFEST_CAPITAL`, `MANIFEST_MARKET`, `MANIFEST_REALARM`, and `MANIFEST_FLAGSHIP` (flagship arm). `PREREGISTRATION.md` and `PREREGISTRATION_AMENDMENTS.md` freeze the design, so any silent change to an input or to the pre-registered plan is detectable before the analysis runs. Post-hoc analyses (the flagship tier arm, the self-consistency curve, and the score validation) are labelled as such in both the manuscript and this document.

4.3 Data sources

All confirmatory inputs are synthetic and self-contained: the 90-item battery is frozen and its Altman Z'' benchmark is independently computable before any model is queried. The real-firm arm uses only public SEC 10-K filings and XBRL company facts (keyless EDGAR; permanent URLs in `ratings_provenance.csv`); the disposable build cache is not shipped and is not needed to reproduce any result. The original capture queried the OpenAI and Google APIs; those responses are frozen under `data/raw/` and `capture/flagship_runs/` and are the only trace of the live phase the reproduction consumes.

5. Code (what it consumes and runs)

5.1 Analysis scripts

All code is in `analysis/`; each module is deterministic (seed 42) and reads only frozen data.

Script	Purpose	Produces
<code>run_analysis.py</code>	confirmatory analysis: OLS Type-II ANOVA variance decomposition (item / axes / provider×item / pure-seed / other), determinism-subgrid permutation test, within/cross-vendor agreement, granularity κ , IG/HY flip	<code>results.json</code> , <code>spec_curves.json</code> , <code>H1_results.json</code> , <code>MANIFEST_ANALYSIS</code>
<code>capital_analysis.py</code>	post-hoc Basel CRE20 capital recast on the frozen panel	<code>results_capital.json</code>
<code>market_analysis.py</code>	Cornaggia-anchored implied-spread translation	<code>results_market.json</code>
<code>addenda.py</code>	supplementary robustness figures/numbers	<code>results_addenda.json</code>
<code>benchmark_validation.py</code>	power / benchmark validation checks	<code>benchmark_validation.*</code> , <code>MANIFEST_VALIDATION</code>

Script	Purpose	Produces
<code>analyze_realarm.py</code>	real-firm arm: fingerprint gate, WATCH-stratum flip-share, TOST equivalence	real-arm results
<code>tier_analysis.py</code>	flagship vs nano/flash IG/HY & band flip on the 12-spec grid, issuer-clustered CIs	<code>results_tier.json</code>
<code>extended_analysis.py</code>	specification-instability score validation (ROC-AUC, held-out & real-issuer), self-consistency curve, ordinal-weighted agreement, per-stratum flips, deployment cost	<code>results_extended.json</code>
<code>build_panel.py</code>	(audit) rebuild <code>panel.parquet</code> from the raw corpus	panel (row-identical)
<code>make_figures.py</code> , <code>make_extended_figures.py</code>	regenerate manuscript figures (optional; needs matplotlib)	<code>paper/ISWA/*.pdf,png</code>

5.2 Pipeline order

`reproduce.sh` runs the modules in dependency order; all JSON outputs are timestamp-free, so re-runs are byte-identical:

```
run_analysis→capital_analysis→market_analysis→addenda→benchmark_validation→analyze_realarm→tier_analysis→extended_analysis
```

6. Results (generated outputs)

Running the pipeline regenerates the following under `results/`. In a clean checkout these already exist; the reproduction overwrites them and confirms they are unchanged.

Output	Contents
<code>results.json</code>	headline confirmatory numbers: variance shares (with issuer-clustered CIs), agreement ladder, IG/HY flip (25.2%, 95% CI 19.7–31.2), determinism permutation $p = 0.001$
<code>spec_curves.json</code>	per-specification accuracy / IG-share for the specification-curve figure
<code>results_tier.json</code>	flagship vs nano/flash flip shares and paired differences
<code>results_extended.json</code>	score validation (held-out ROC-AUC 0.94, CI 0.86–0.99; real-issuer 0.70), self-consistency curve ($k = 1 \dots 15$), Krippendorff α , per-stratum flips, deployment cost
<code>results_capital.json</code> , <code>results_market.json</code> , <code>results_addenda.json</code>	economic translations (capital recast, implied spread, addenda)
<code>H1_results.json</code>	real-firm arm hypothesis-test outputs
<code>benchmark_validation.{json, csv}</code>	power / benchmark validation
<code>results/exhibits/</code>	regenerated tables

7. Reproduction and verification

`reproduce.sh` performs, and exits non-zero if any stage fails:

1. **Integrity.** Each frozen stage carries a SHA-256 manifest (`manifest/`); the raw-corpus freeze rejects any capture file above the pre-set error threshold (observed non-substantive rate 0.08%, 7 of 8,640 confirmatory cells, dropped listwise).
2. **Regeneration.** The full analysis (Section 5.2) runs from the frozen inputs only.
3. **Byte-identical re-run.** Re-running `reproduce.sh` regenerates every `results/results_*.json` byte-for-byte (verified); the results of record are the committed JSON files.

Determinism. All stochastic steps use fixed seeds (analysis seed 42; issuer-clustered bootstraps use fixed-seed resamples), so confidence intervals are exactly reproducible. *Note:* Parquet serialization is not byte-deterministic, so a rebuilt `panel.parquet` may hash differently while containing row-identical data; the confirmatory analysis reads the released frozen panel, so every reported number reproduces exactly.

Environment. Pinned analysis environment: `numpy==2.2.6`, `pandas==2.2.3`, `pyarrow==24.0.0`, `statsmodels==0.14.6`, `scipy==1.15.3`, `patsy==1.0.2`, on Python 3.10 (exact pins in `requirements.txt`). Figures are optional and need `matplotlib`, which is not part of the pinned analysis dependencies.

8. Live data collection (not part of the reproduction)

`capture/` documents how the model-response corpora were originally collected: `collect.py` drives the main confirmatory grid; `run_arm.py` and `realarm_collect.py` run the real-firm arm; `run_flagship.py` runs the flagship model-tier arm; `freeze_flagship.py` writes its SHA-256 manifest; `fingerprint_pilot.py` runs the decontamination gate; `list_models.py` enumerates served models. This phase requires internet access and OpenAI/Google API keys, incurs cost, and is intentionally excluded from the reproducible core. API keys are never distributed (`.gitignore` blocks `keys.env.txt` and any `API Keys/` folder); all reported results derive only from the frozen corpora.

9. Licensing and data availability

Code is released under the MIT Licence (see `LICENSE`); data, analysis outputs, and text are released under CC-BY-4.0. The confirmatory data are synthetic; the real-firm arm uses only public SEC filings (anonymized and perturbed). The permanent citable archive is the Zenodo deposit at `10.5281/zenodo.21953934`, which reproduces every reported number, table, and figure offline, deterministically, and at zero cost.